In this section I introduce important reinforcement learning algorithms while pushing the theoretical results to the next chapter. The central problem is how do we move from estiamating V_π and V_* when we do not have a model.

0.1 The Classical Synthesis

0.1.1 Monte Carlo Estimation

When $P(s'|s,a)$ and $r(s,a)$ are unknown, the obvious approach is to use Monte Carlo (MC) to approximate them. These methods estimate value functions from sampled episodes $(s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_T)$. The realized return from state s_t is:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1} \tag{1}$$

First-visit MC prediction averages G_t over episodes for each state s, counting only its first occurrence per episode. Each first-visit return is an independent draw from the return distribution, so the sample mean converges almost surely to $V^\pi(s)$ by the strong law of large numbers (Sutton and Barto, 2018). An incremental update is,

$$V(s) \leftarrow V(s) + \alpha[G_t - V(s)]$$

For MC control, we can estimate action-values $Q(s,a)$ by averaging first-visit returns from each state-action pair, then improve the policy greedily: $\pi(s) = argmax_a\, Q(s,a)$. Under exploring starts (every (s,a) pair begins an episode infinitely often), this alternation converges to Q^*. [1]

0.1.2 Sutton (1988)

Monte Carlo has two limitations. The agent must wait for episode termination to compute G_t, ruling out continuing tasks. And G_t is unbiased ($\mathbb{E}[G_t \mid S_t = s] = V^\pi(s)$) but high-variance, because it sums random rewards over the entire trajectory.

[1] Tsitsiklis (2002) proved convergence even when the policy improves after every episode rather than waiting for complete evaluation. In practice, exploring starts is infeasible, so on-policy variants use ε-greedy exploration instead. These converge to Q^* provided the exploration schedule satisfies the greedy-in-the-limit with infinite exploration (GLIE) condition: every state-action pair is visited infinitely often, and $\varepsilon_t \to 0$ so the policy converges to greedy in the limit.

Sutton (1988) proposed temporal difference (TD) learning to fix both problems. TD(0) replaces the full return G_t with a one-step target:

$$V(s_t) \leftarrow V(s_t) + \alpha \left[r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \right] \tag{2}$$

The target $r_{t+1} + \gamma V(s_{t+1})$ depends on one random reward and one random transition, so its variance is low. The cost is bias: $V(s_{t+1})$ is the agent's current estimate, not the true value. This is "bootstrapping".[2] As V improves, the bias shrinks; the low variance persists regardless. Sutton demonstrated this tradeoff on a five-state random walk where TD(0) converged faster than Monte Carlo with less data.

The general TD(λ) update interpolates between these extremes through an eligibility trace:[3]

$$V(s_t) \leftarrow V(s_t) + \alpha \, \delta_t \, e_t(s) \tag{3}$$

where $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ is the TD error and $e_t(s) = \gamma\lambda \, e_{t-1}(s) + \mathbb{1}\{s = s_t\}$ is the eligibility trace for state s. Setting $\lambda = 0$ yields TD(0); setting $\lambda = 1$ recovers Monte Carlo returns. Intermediate λ trades off variance against bias. [4]

0.1.3 Watkins (1989)

TD(0) learns value functions $V(s)$, but converting these to actions (the control problem) still requires knowing transition probabilities. Given $V(s')$ for all successor states, the agent needs to know which action leads to which successor.

Watkins and Dayan (1992), formalizing Watkins's 1989 PhD thesis, provided the solution. Instead of learning $V(s')$ learn $Q(s, a)$ (the action value function or the "quality" of actions function) directly, the expected return from taking action a in state s and then behaving optimally. The optimal policy is then $\pi^*(s) = argmax_a \, Q^*(s, a)$, requiring no model to act. The Bellman optimality equation provides the fixed-point condition:

$$Q^*(s, a) = \mathbb{E}\left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right] \tag{4}$$

[2] Bootstrapping means updating value estimates using other value estimates rather than observed returns.

[3] The eligibility trace records which states were recently visited, allowing credit assignment to propagate backward in time. States visited more recently receive stronger updates when the TD error is observed.

[4] Dayan (1992) proved convergence of TD(λ) for general λ in the tabular case; Jaakkola et al. (1994) gave a unified stochastic approximation proof covering both TD and Q-learning; Tsitsiklis and Van Roy (1997) extended the analysis to linear function approximation.

Q-learning achieves this via incremental updates:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \tag{5}$$

Q-learning converges to Q^* under standard regularity conditions (Section **??**). The maximization over a' makes Q-learning off-policy:[5] the update target uses the greedy action at the next state regardless of the action actually taken. Therefore the agent can follow an ε-greedy exploration strategy or even a fully random policy, while learning about the optimal policy directly.

0.1.4 Williams (1992)

Value-based methods learn an action-value function and derive a policy from it. Williams (1992) derived an alternative, policy gradients, that optimize the policy directly by gradient ascent on expected return:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t \right] \tag{6}$$

where $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k+1}$ is the discounted return from time t. The log-derivative trick allows the gradient to be estimated from sampled trajectories without differentiating through the environment dynamics.

Consider a robot arm that must apply a continuous torque $a \in \mathbb{R}$ to reach a target angle. Q-learning requires computing $\max_a Q(s, a)$ at every update, which becomes a nested optimization problem [6] when the action space is continuous. A policy gradient method sidesteps the issue. Parameterize the policy as a Gaussian $\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s),\, \sigma^2_\theta(s))$,[7] sample an action, observe the return, and update θ by REINFORCE. No maximization over actions is required. Virtually all continuous-control results in reinforcement learning descend from the policy gradient framework rather than from Q-learning for this reason.

[5] Off-policy learning means the agent learns about a policy different from the one generating data. Q-learning learns about the greedy policy $\pi^*(s) = \mathit{argmax}_a\, Q(s, a)$ while collecting data with an exploratory policy. This exploratory policy needs only to be sufficiently "exploratory" and admits a wide range of policies; including a fully random policy.

[6] In continuous action spaces, $\max_a Q(s, a)$ has no closed-form solution in general and must be solved numerically at every Bellman update. Discretizing a d-dimensional action space on a grid of m points per dimension costs $O(m^d)$ evaluations per update step.

[7] The Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ has density $(2\pi\sigma^2)^{-1/2} \exp(-(a-\mu)^2/2\sigma^2)$. Here $\mu_\theta(s)$ and $\sigma_\theta(s)$ are neural network outputs parameterizing the policy mean and standard deviation.

0.1.5 Tesauro (1994)

Tesauro (1994)'s TD-Gammon demonstrated that temporal difference learning with neural network function approximation could achieve expert-level play in a domain with approximately 10^{20} legal positions[8].

Backgammon was far beyond tabular methods, yet TD-Gammon trained a feedforward neural network to estimate the probability of winning from any board position. A hidden layer of 80 sigmoid units fed a single sigmoid output:

$$\hat{V}(s) = \sigma\Big(\mathbf{w}^\top \sigma(W\mathbf{x}(s) + \mathbf{b}) + c\Big) \tag{7}$$

where σ is the logistic sigmoid, W is the input-to-hidden weight matrix, and $\mathbf{w}$ is the hidden-to-output weight vector. The network was trained by self-play using TD(λ) with $\lambda = 0.7$. After each move from position s_t to s_{t+1}, the weights $\boldsymbol{\theta}$[9] were updated by

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\Big[\hat{V}(s_{t+1}) - \hat{V}(s_t)\Big]\,\mathbf{e}_t \tag{8}$$

where the eligibility trace $\mathbf{e}_t = \sum_{k=1}^{t} \lambda^{t-k} \nabla_{\boldsymbol{\theta}} \hat{V}(s_k)$ accumulates exponentially decayed gradients of past predictions.[10] At game's end, $\hat{V}(s_{t+1})$ is replaced by the outcome $z \in \{0, 1\}$.

Self-play proceeds as follows. A single neural network $\hat{V}$ serves as the evaluation function for both players. At each turn, the current player rolls the dice, which determines the set of legal moves. For each legal move, the network evaluates the resulting board position $\hat{V}(s')$. The player selects the move that maximizes $\hat{V}(s')$ from its own perspective. The opponent then does the same using the identical network. After each move from position s_t to s_{t+1}, the weights are updated by the TD(λ) rule using the error $\hat{V}(s_{t+1}) - \hat{V}(s_t)$, which pushes earlier predictions toward later ones. When the game ends, the terminal signal $z \in \{0, 1\}$ replaces $\hat{V}(s_{t+1})$, grounding the entire chain of predictions in the actual outcome. As the network improves, it generates stronger play on both sides,

[8] The state $\mathbf{x}(s)$ was a 198-dimensional binary encoding of the raw board (four units per board point per player indicating checker counts, plus bar and borne-off counts). The output was a four-component vector estimating probabilities of each game outcome (White/Black $\times$ normal win/gammon), and the move maximizing expected outcome among all legal moves was selected at each step.

[9] $\boldsymbol{\theta}$ denotes the full vector of network weights and biases, generalizing the scalar θ used for policy parameters in earlier sections.

[10] In the neural network case, the eligibility trace $\mathbf{e}_t \in \mathbb{R}^{|\boldsymbol{\theta}|}$ is a vector accumulating exponentially-weighted gradients, extending the scalar state-based trace to parameter space.

which produces harder training games, which in turn drives further improvement. The dice rolls ensure that the network encounters a wide variety of board positions without requiring an explicit exploration mechanism[11].

0.1.6 SARSA (1994)

Q-learning learns the optimal action-value function regardless of the policy generating experience. This off-policy property is useful but introduces complications when combined with function approximation. Rummery and Niranjan (1994) introduced SARSA as an on-policy[12] alternative that learns the value of the policy actually being followed. The name derives from the quintuple $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$ used in each update:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right] \tag{9}$$

The key difference from Q-learning is that SARSA bootstraps from the action a_{t+1} actually taken at the next state, rather than the greedy action $argmax_{a'}\, Q(s_{t+1}, a')$. This makes the algorithm on-policy, since the target depends on the behavior policy generating the data.

If the agent follows an ε-greedy policy, SARSA converges to $Q^{\varepsilon\text{-greedy}}$, not Q^*[13] policies and standard step-size conditions (Section **??**). This distinction matters when exploration is costly. Consider the cliff-walking problem, where an agent must traverse a gridworld with a cliff along one edge. The optimal path runs along the cliff edge (shortest route), but the ε-greedy policy occasionally falls off. Q-learning learns the optimal path because it evaluates the greedy policy; the agent falls off during learning but the Q-values reflect the optimal route. SARSA learns a safer path further from the cliff because it evaluates the actual exploratory policy; it accounts for the fact that exploration sometimes leads to catastrophic states.

[11] TD-Gammon 1.0, trained on 300,000 self-play games, played at a strong intermediate level. Version 1.0 trained on 300,000 self-play games with 80 hidden units and no lookahead search; version 2.1 trained on 1,500,000 games with 2-ply search, reducing its average loss rate against world-class human players from -0.25 to -0.02 points per game. Version 2.1 achieved near-parity with Bill Robertie, a former world champion. The program discovered novel positional strategies subsequently adopted by the human backgammon community.

[12] On-policy methods learn about the policy currently generating data, in contrast to off-policy methods (see footnote on Q-learning above) which learn about a different target policy.

[13] SARSA converges to Q^* under GLIE (greedy in the limit with infinite exploration)A GLIE schedule explores all state-action pairs infinitely often but converges to the greedy policy asymptotically. The standard ε-greedy policy with $\varepsilon_t \to 0$ is one such schedule.

0.1.7 Baird (1995)

Baird (1995) constructed a six-state star MDP demonstrating divergence of Q-learning with linear function approximation. The MDP has five outer states that all transition to a single inner state under the target policy. The off-policy behavior samples states uniformly. With linear function approximation, the weights grow without bound under repeated Q-learning updates. The source of instability is the interaction of three components, namely bootstrapping (updating from estimated values rather than observed returns), off-policy learning (training on data from a different policy than the target), and function approximation (representing the value function with a parameterized model). Sutton and Barto (2018) later named this the deadly triad. Any two components can be combined safely; all three together permit divergence. The mechanism underlying this instability is analyzed in Section **??**.

This result explains the asymmetry between Tesauro's success and Baird's failure. TD-Gammon used bootstrapping and function approximation but was on-policy, with training data coming from the same self-play policy whose value was being estimated. Baird's counterexample used all three components and diverged. Baird also proposed a constructive solution, namely residual gradient algorithms that perform gradient descent on the mean-squared Bellman residual, guaranteeing convergence at the cost of a different fixed point.

0.1.8 Actor-Critic Methods (2000)

The idea of maintaining both a policy (actor) and a value function (critic) dates to Barto et al. (1983), who used a two-component system to solve the pole-balancing task. The actor selected actions; the critic predicted future reinforcement and generated training signals for the actor. This architecture addressed a limitation of pure policy gradient methods. REINFORCE estimates gradients from Monte Carlo returns, which are unbiased but have high variance because returns depend on the full trajectory of stochastic transitions and rewards. TD methods use bootstrapped targets $r + \gamma V(s')$, which have lower variance but introduce bias when V is approximate.

Actor-critic methods combine both ideas. The critic learns a value function $V(s)$ by

TD updates:

$$V(s_t) \leftarrow V(s_t) + \alpha_c \, \delta_t, \quad \delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \tag{10}$$

The actor updates the policy using the TD error as a sample of the advantage:

$$\theta \leftarrow \theta + \alpha_a \nabla_\theta \log \pi_\theta(a_t|s_t) \, \delta_t \tag{11}$$

The TD error δ_t estimates $A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$, the advantage of action a_t over the average action.[14] Positive advantages indicate the action was better than expected; negative advantages indicate it was worse.

Konda and Tsitsiklis (2000) provided the first convergence proof for actor-critic algorithms with function approximation, showing convergence to a local optimum under two-timescale learning rates ($\alpha_c \gg \alpha_a$) and a compatibility condition on the critic architecture (Section **??**).

0.1.9 Natural Policy Gradient (2001)

Standard gradient descent treats all parameter directions equally, but policy parameters define probability distributions whose natural geometry is not Euclidean. Kakade (2001) introduced the natural policy gradient, which measures progress in distribution space rather than parameter space. The update uses the Fisher information matrix $F(\theta)$:

$$F(\theta) = \mathbb{E}_{s \sim d^\pi, a \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s)^\top \right] \tag{12}$$

The natural gradient is:

$$\tilde{\nabla}_\theta J(\theta) = F(\theta)^{-1} \nabla_\theta J(\theta) \tag{13}$$

This direction is invariant to reparameterization of the policy. In the tabular softmax[15] case, a single natural gradient step with unit step size recovers one step of exact policy iteration (Section **??**).

The computational bottleneck is inverting $F(\theta) \in \mathbb{R}^{d \times d}$. Practical implementations use conjugate gradient methods to solve $F(\theta)x = \nabla_\theta J(\theta)$ without forming F explicitly.

[14] That δ_t is an unbiased estimate of the advantage follows from the policy gradient theorem (Sutton et al., 2000).

[15] The softmax function maps a vector $\mathbf{z}$ to a probability distribution: $\text{softmax}(z_i) = \exp(z_i) / \sum_j \exp(z_j)$. A softmax policy parameterizes action probabilities as $\pi_\theta(a|s) = \text{softmax}(\theta_{s,a})$.

This approach was later scaled to deep neural networks by TRPO and PPO.

0.2 The Deep Learning Era

0.2.1 Deep Q-Networks (2015)

The two decades after TD-Gammon saw limited success in applying neural networks to reinforcement learning. Attempts to replicate Tesauro's approach in chess and Go did not achieve comparable results, though progress continued in robotics and other domains. Mnih et al. (2015) changed this by training a single convolutional neural network[16] to play 49 Atari 2600 games directly from pixel inputs ($210 \times 160 \times 3$) and a scalar score, using no game-specific features.

The architecture processed four consecutive frames through three convolutional layers and a fully connected layer. The network $Q(s, a; \theta)$ was trained to minimize the squared temporal difference error:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \tag{14}$$

Two innovations stabilized learning. Experience replay (Lin, 1992) stored transitions (s, a, r, s') in a buffer $\mathcal{D}$ holding 10^6 transitions and sampled uniformly for training. This breaks the temporal correlation between consecutive updates that would otherwise cause the network to overfit to recent experience. A target network used a frozen copy of parameters θ^-,[17] updated to match θ only every $C = 10{,}000$ steps, so the regression target does not shift with each gradient step.

DQN exceeded human-level performance on 29 of 49 games using a single architecture and hyperparameters. The same network that learned Breakout (where the agent must track a ball and position a paddle) also learned Space Invaders (where the agent must track multiple enemies and time shots). Games requiring long-horizon planning or sparse rewards, such as Montezuma's Revenge (where the agent must navigate rooms and collect keys before receiving any reward), remained difficult.

[16] A convolutional neural network applies learned spatial filters to detect local patterns in grid-structured data, commonly used for image inputs.

[17] The notation θ^- denotes target network parameters, a lagged copy of θ updated periodically. This stabilizes learning by keeping the regression target fixed during gradient steps.

0.2.2 TRPO and PPO (2015, 2017)

Policy gradient methods suffer from a practical instability. A single large gradient step can move the policy into a region of parameter space where performance collapses, and recovery may be slow or impossible. The objective $J(\theta)$ is not well approximated by its linear Taylor expansion when the step size is large relative to the curvature of the policy space.

Schulman et al. (2015) addressed this with Trust Region Policy Optimization (TRPO). TRPO solves the constrained optimization problem

$$\max_{\theta} \; L_{\theta_{\text{old}}}(\theta) \quad \text{subject to} \quad \bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta) \leq \delta \tag{15}$$

where L is a surrogate objective based on the advantage function $A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$.[18] The KL constraint prevents the policy from moving so far in one update that performance collapses and recovery becomes impossible.

Schulman et al. (2017) proposed Proximal Policy Optimization (PPO) as a simpler alternative. PPO replaces the KL constraint with a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min\Big(r_t(\theta)\hat{A}_t, \; \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t \Big) \right] \tag{16}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio. The clipping removes the incentive for $r_t(\theta)$ to move outside the interval $[1-\varepsilon, 1+\varepsilon]$, penalizing large policy updates without explicitly computing a divergence measure.

PPO outperformed A2C, TRPO, and the cross-entropy method on continuous control benchmarks and achieved the highest average reward on 30 of 49 Atari games among the methods tested. PPO became the default policy optimization algorithm for large-scale RL applications, demonstrating that constraining the magnitude of policy updates is essential for stable optimization.

0.2.3 Soft Actor-Critic (2018)

Haarnoja et al. (2018) introduced Soft Actor-Critic (SAC), which adds entropy regularization to the actor-critic framework. The agent maximizes expected return plus an

[18] The Kullback-Leibler divergence $D_{\text{KL}}(p\|q) = \sum_x p(x)\log(p(x)/q(x))$ measures statistical distance between distributions. The bar denotes expectation over states: $\bar{D}_{\text{KL}} = \mathbb{E}_s[D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s)\|\pi_\theta(\cdot|s))]$.

entropy bonus:

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t \left(r_t + \tau \mathcal{H}(\pi_\theta(\cdot|s_t))\right)\right] \tag{17}$$

where $\mathcal{H}(\pi) = -\sum_a \pi(a) \log \pi(a)$ is the entropy and $\tau > 0$ is a temperature parameter. The entropy bonus encourages exploration by penalizing deterministic policies. The optimal policy under this objective is softmax in the Q-values: $\pi^*(a|s) \propto \exp(Q^*(s,a)/\tau)$, connecting to discrete choice models in econometrics.

SAC maintains two Q-networks (to reduce overestimation bias) and a policy network. The soft Bellman operator for the critic is:

$$(T^\pi Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s'}\left[V(s')\right], \quad V(s) = \mathbb{E}_{a\sim\pi}[Q(s,a) - \tau \log \pi(a|s)] \tag{18}$$

SAC is off-policy (using experience replay), handles continuous actions naturally, and achieves state-of-the-art sample efficiency on continuous control benchmarks. The entropy regularization provides automatic exploration without ε-greedy schedules.

0.2.4 AlphaGo Zero (2017)

The game of Go has approximately 10^{170} legal positions and a branching factor of roughly 250, far beyond the reach of brute-force search. Hand-crafted evaluation functions, which had succeeded in chess, failed here because positional concepts like influence, territory, and group viability are holistic and contextual. Monte Carlo tree search (MCTS) had achieved amateur-level play by using random simulations to estimate position values, but progress had stalled below professional strength. Silver et al. (2016) broke through by combining supervised learning from 30 million human expert positions, reinforcement learning via self-play, and MCTS with learned value and policy networks; the resulting system defeated Lee Sedol four games to one in March 2016. A year later, Silver et al. (2017) showed that none of the human data was necessary.

AlphaGo Zero uses a single neural network $f_\theta(s) = (\mathbf{p}, v)$ that takes a board position s and outputs both a policy vector $\mathbf{p}$ over legal moves and a scalar value v estimating the probability of winning.[19] The input representation consists of 17 binary planes on the 19×19 board, encoding the raw game state without any hand-crafted features: eight

[19] A convolutional neural network applies small learned filters that slide across a grid input, detecting local patterns regardless of position on the board. The same filter that recognizes a capturing pattern in one corner will recognize it in another.

planes for Black's stone positions over the last eight moves, eight for White's, and one indicating which color plays next.[20] The board's fixed grid structure makes convolutional networks effective, since spatial patterns learned in one region transfer to others. The architecture uses residual blocks,[21] which allow training of very deep networks.

During play, each move is selected by running MCTS, which conducts 1,600 simulated games from the current position to estimate move quality. Each simulation proceeds in four phases, illustrated in Figure 1. In the selection phase, the algorithm starts from the current position and traverses the partially built search tree by choosing at each node the action that maximizes $Q(s,a) + c_{\text{puct}} \cdot P(s,a) \cdot \sqrt{\sum_b N(s,b)} \,/\, (1 + N(s,a))$, where $Q(s,a)$ is the current average value of action a, $P(s,a)$ is the prior probability from the neural network, and $N(s,a)$ is the visit count.[22] In the expansion and evaluation phase, when the traversal reaches a position not yet in the tree, the neural network evaluates it in a single forward pass,[23] producing a policy vector $\mathbf{p}$ and a value estimate v; the policy initializes prior probabilities $P(s',a) = p_a$ for each child edge. In the backup phase, the value v propagates back up the traversed path, incrementing each edge's visit count $N(s,a)$ and updating its mean value $Q(s,a)$. After all 1,600 simulations, the algorithm selects the move with the highest visit count at the root.

The training loop generates self-play games. At each board position s_t during a game, the program runs MCTS to produce improved move probabilities $\boldsymbol{\pi}_t$, where $\pi_t(a) \propto N(s_t,a)^{1/\tau}$ and τ is a temperature parameter controlling exploration.[24] A move a_t is sampled from $\boldsymbol{\pi}_t$ and played. At game end, the outcome $z \in \{-1,+1\}$ is recorded. Each position becomes a training triple $(s_t, \boldsymbol{\pi}_t, z)$, and the network parameters are updated to minimize

$$\ell(\theta) = (z - v)^2 - \boldsymbol{\pi}^\top \log \mathbf{p} + c\|\theta\|^2 \tag{19}$$

[20] The history planes allow the network to detect ko situations (positions that cannot be immediately repeated) and infer the trajectory of play without any explicit feature engineering.

[21] A residual block computes $\mathbf{x} + g(\mathbf{x})$ rather than just $g(\mathbf{x})$, where g is a learned transformation. The skip connection allows gradients to flow through very deep networks without vanishing. AlphaGo Zero's 40-block architecture has 79 parameterized layers.

[22] The constant c_{puct} controls exploration. Actions visited often have well-estimated Q values but a shrinking exploration bonus; rarely visited actions have uncertain values but a large bonus. This is a continuous analogue of the upper confidence bound (UCB) strategy from bandit theory.

[23] A forward pass is one evaluation of the neural network on an input, producing outputs without any parameter updates.

[24] Early in the game ($t \leq 30$), $\tau = 1$ so moves are sampled proportionally to visit counts, encouraging diverse openings. Later, $\tau \to 0$ and the most-visited move is selected deterministically. Dirichlet noise is also added to root priors, $P(s,a) = (1-\varepsilon)p_a + \varepsilon\eta_a$ with $\eta \sim \text{Dir}(0.03)$, ensuring all legal moves can be explored despite strong network priors.

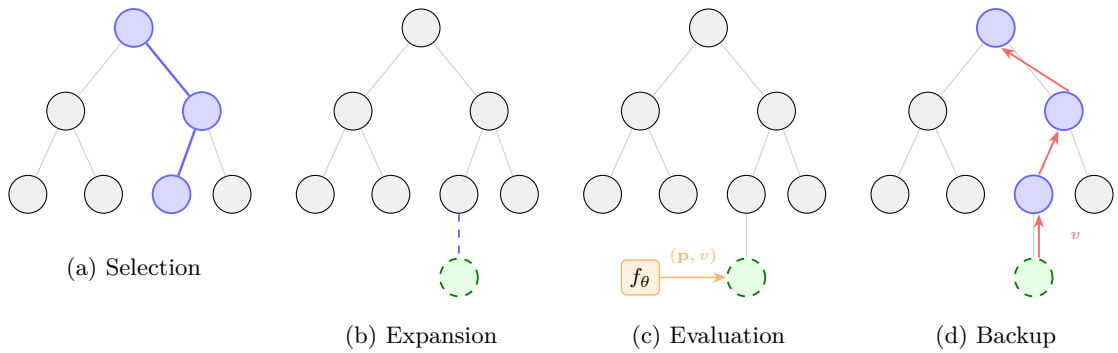

Figure 1: The four phases of a single MCTS simulation in AlphaGo Zero. (a) Selection traverses the tree from the root, choosing at each node the action maximizing a UCB-like score balancing exploitation (Q) and exploration (P/N). (b) Expansion adds a new leaf node when the traversal reaches an unexplored position. (c) The neural network f_θ evaluates the new position, producing move priors $\mathbf{p}$ and a value estimate v. (d) Backup propagates v along the traversed path, updating mean values $Q(s, a)$ and visit counts $N(s, a)$ at each edge.

where the first term is a value prediction loss, the second is a policy cross-entropy loss,[25] and the third is L_2 regularization.[26] The key mechanism is a virtuous cycle. MCTS serves as a policy improvement operator, since the search probabilities $\boldsymbol{\pi}$ are stronger than the raw network outputs $\mathbf{p}$. Training the network to match $\boldsymbol{\pi}$ distills the search improvements back into the network, and the improved network in turn produces better MCTS. After 72 hours of self-play on 4 TPUs, AlphaGo Zero surpassed all previous versions, including the one that defeated Lee Sedol, and discovered novel strategies not previously seen in human play.

Go was well-suited to this architecture. Its fixed 19×19 board maps naturally to convolutional networks, its perfect information and deterministic transitions make MCTS's tree structure exact, and the binary game outcome provides an unambiguous training signal. Igami (2020) interprets the architecture in econometric terms, where the policy network is a conditional choice probability (CCP) estimator, the value network is a conditional value function (CVF) estimator, and the system performs CCP estimation and forward simulation jointly, connecting to the approach of Hotz and Miller (1993) in dynamic discrete choice.

[25] The cross-entropy loss $-\boldsymbol{\pi}^\top \log \mathbf{p}$ measures how well the predicted distribution $\mathbf{p}$ matches the target $\boldsymbol{\pi}$; it equals zero when the distributions are identical.

[26] L_2 regularization penalizes the squared magnitude of parameters, $c\|\theta\|^2$, analogous to ridge regression in econometrics.

References

Leemon Baird. Residual algorithms: Reinforcement learning with function approximation. In *Proceedings of the Twelfth International Conference on Machine Learning*, pages 30–37. Morgan Kaufmann, 1995.

Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(5):834–846, 1983.

Peter Dayan. The convergence of TD(λ) for general λ. *Machine Learning*, 8(3–4):341–362, 1992.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning*, pages 1861–1870, 2018.

V. Joseph Hotz and Robert A. Miller. Conditional choice probabilities and the estimation of dynamic models. *The Review of Economic Studies*, 60(3):497–529, 1993.

Mitsuru Igami. Artificial intelligence as structural estimation: Deep blue, bonanza, and alphago. *The Econometrics Journal*, 23(3):S1–S24, 2020.

Tommi Jaakkola, Michael I. Jordan, and Satinder P. Singh. On the convergence of stochastic iterative dynamic programming algorithms. In *Advances in Neural Information Processing Systems 6*, pages 703–710. Morgan Kaufmann, 1994.

Sham M. Kakade. A natural policy gradient. *Advances in Neural Information Processing Systems*, 14, 2001.

Vijay R. Konda and John N. Tsitsiklis. Actor-critic algorithms. In *Advances in Neural Information Processing Systems 12*, pages 1008–1014. MIT Press, 2000.

Long-Ji Lin. *Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching*. PhD thesis, Carnegie Mellon University, 1992. Also published in *Machine Learning*, 8(3–4):293–321, 1992.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

G. A. Rummery and M. Niranjan. On-line q-learning using connectionist systems. Technical report, Cambridge University, 1994.

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. *Proceedings of the 32nd International Conference on Machine Learning*, pages 1889–1897, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017.

Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988. doi: 10.1023/A:1022633531479.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, 2nd edition, 2018.

Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gra-

dient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 2000.

Gerald Tesauro. Td-gammon, a self-teaching backgammon program, achieves master-level play. *Neural Computation*, 6(2):215–219, 1994.

John N. Tsitsiklis. On the convergence of optimistic policy iteration. *Journal of Machine Learning Research*, 3:59–72, 2002.

John N. Tsitsiklis and Benjamin Van Roy. Analysis of temporal-difference learning with function approximation. In *Advances in Neural Information Processing Systems 9 (NIPS)*, 1997.

Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4): 279–292, 1992.

Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.